

Listas e tuplos: guardar e processar conjuntos de valores

Atividade 1 · Índices, fatias e alteração

Questões de autoavaliação

1. Registrar fatia e lista final. Explicar o efeito de append no comprimento e distinguir índice e valor.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original: primeiro e último valores 8 e 18; fatia [12, 15, 7]; lista final [10, 12, 15, 7, 18, 16]. Na experiência, a fatia `notas[:2]`, antes das alterações, é [8, 15, 18]. A lista final fica [10, 12, 15, 7, 18, 20], com comprimento 6.

`append` acrescenta um elemento. O índice é a posição; o valor é o conteúdo dessa posição.

```
notas = [8, 12, 15, 7, 18]
print(notas[0], notas[-1])
print(notas[:2])
notas[0] = 10
notas.append(16)
notas[-1] = 20
print(notas)
print(len(notas))
```

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 2 · Dois nomes para a mesma lista

Questões de autoavaliação

1. Comparar os estados das listas antes e depois de copiar. Explicar quais os nomes que partilham a mesma lista.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Antes da alteração: a e b valem [99, 20], c vale [10, 20, 30]. Com `b = a.copy()`: a fica [10, 20], b fica [99, 20], c fica [10, 20, 30].

Na primeira versão, a e b referem-se à mesma lista. Na segunda, b recebe uma lista distinta. Reexecutar a célula inteira é necessário para repor o estado inicial.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Listas e tuplos: guardar e processar conjuntos de valores

Atividade 3 · Selecionar e resumir

Questões de autoavaliação

1. Registrar os resultados, justificar o tratamento da lista vazia e distinguir média da turma e média dos selecionados.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original: aprovados [12, 15, 18], quantidade 3, média da turma 12.0. Selecionando notas inferiores a 10: [8, 7], quantidade 2. A média da turma continua 12.0; a média dos selecionados seria 7.5.

```
notas = []
reprovados = [n for n in notas if n < 10]
print('Reprovados:', reprovados)
print('Quantidade:', len(reprovados))
if notas:
    print('Média da turma:', sum(notas) / len(notas))
else:
    print('Sem notas')
```

Com lista vazia, a seleção é [] e a quantidade zero. A divisão original causa ZeroDivisionError; o if impede-a.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 4 · Tuplos e representação gráfica

Questões de autoavaliação

1. Apresentar o novo resumo e descrever o gráfico. Justificar a criação de um novo tuplo em vez de alterar os elementos do anterior.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original: resumo (7, 18). Após acrescentar 10: a média é 70 / 6, aproximadamente 11.6667, o mínimo é 7 e o máximo 18. O novo resumo contém esses três valores. O gráfico passa de cinco para seis barras; a última tem altura 10.

```
notas = [8, 12, 15, 7, 18]
notas.append(10)
resumo = (sum(notas) / len(notas), min(notas), max(notas))
print(resumo)
```

Um tuplo não permite substituir os seus elementos. Associar resumo a um novo tuplo é permitido. O restante código do gráfico mantém-se.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Listas e tuplos: guardar e processar conjuntos de valores

Atividade 5 · Aplicação integrada

Questões de autoavaliação

1. Explicar a organização do algoritmo e as decisões tomadas.
2. Comparar resultados previstos e observados em todos os testes pedidos.
3. Identificar uma falha possível e indicar como um teste a poderia revelar.

Resolução e verificação

```
def analisar_notas(notas):  
    if not notas:  
        return None  
    return (sum(notas) / len(notas), min(notas), max(notas))
```

```
notas = [8, 12, 15, 7, 18]  
resumo = analisar_notas(notas)  
if resumo is None:  
    print('Sem notas')  
else:  
    print('Resumo:', resumo)  
aprovados = [n for n in notas if n >= 10]  
print('Aprovados:', aprovados)  
print('Lista original:', notas)
```

Lista principal: resumo (12.0, 7, 18), aprovados [12, 15, 18]. [10]: (10.0, 10, 10), aprovados [10]. []: a função devolve None, mostra Sem notas e aprovados []. A função não altera a lista recebida.

Listas e tuplos: guardar e processar conjuntos de valores

Atividade 6 · Remover elementos durante um percurso

Questões de autoavaliação

1. Que lista resulta do código inicial e porquê?
2. Que resultados são esperados após a correção?
3. Como confirmar que a lista original foi preservada?

Resolução e verificação

```
notas = [7, 8, 12, 9, 15]
aprovadas = []
for nota in notas:
    if nota >= 10:
        aprovadas.append(nota)
print('Selecionadas:', aprovadas)
print('Original:', notas)
```

Inicial: [8,12,15]. Ao remover 7, o 8 desloca-se para uma posição já ultrapassada pelo percurso. Correto: [12,15], [], [] e [10,20]. A lista original continua [7,8,12,9,15]; pode guardar-se uma cópia antes da chamada e comparar depois.

Listas e tuplos: guardar e processar conjuntos de valores

Atividade 7 · Cópias de listas com listas internas

Questões de autoavaliação

1. Por que a lista original muda apesar de copy()?
2. Que objetos precisam de ser copiados neste exemplo?
3. Quais os estados finais se a cópia receber 20 na primeira posição e 0 na última?

Resolução e verificação

```
turmas = [[10, 12], [8, 15]]
cópia = [linha.copy() for linha in turmas]
cópia[0][0] = 20
cópia[1][1] = 0
print('Original:', turmas)
print('Cópia:', cópia)
```

Inicialmente ambas mostram `[[20,12],[8,15]]`, pois partilham as linhas. Com cópia de cada linha, original `[[10,12],[8,15]]`; cópia `[[20,12],[8,0]]`. É suficiente para esta estrutura de duas dimensões com números; não é uma solução geral para qualquer profundidade de objetos mutáveis.

Listas e tuplos: guardar e processar conjuntos de valores

Atividade 8 · Analisar uma sequência sem alterar os dados

Questões de autoavaliação

1. Quais os resumos e listas sem repetidos nos quatro casos?
2. Por que acima da média exige > e não >=?
3. Como a solução distingue remoção de duplicados e ordenação?

Resolução e verificação

```
def resumir(valores):  
    if not valores:  
        return None  
    media = sum(valores) / len(valores)  
    acima = 0  
    for valor in valores:  
        if valor > media:  
            acima += 1  
    return (media, min(valores), max(valores), acima)  
  
def sem_repetidos(valores):  
    resultado = []  
    for valor in valores:  
        if valor not in resultado:  
            resultado.append(valor)  
    return resultado  
  
valores = [10, 12, 10, 18, 0]  
antes = valores.copy()  
print(resumir(valores))  
print(sem_repetidos(valores))  
assert valores == antes  
assert sem_repetidos(valores) == [10, 12, 18, 0]
```

Resultados: (10.0,0,18,2) e [10,12,18,0]; (5.0,5,5,0) e [5]; (7.0,7,7,0) e [7]; None e []. A comparação é estrita. A ordem de primeira ocorrência é conservada, sem ordenar. Os valores iguais à média não contam como acima.

Critério de autoavaliação: 2 pontos quando a implementação cumpre as regras, passa os testes pedidos e as respostas explicam os resultados; 1 ponto se falta parte da solução, dos testes ou da justificação; 0 pontos se a tarefa permanece por resolver. Aplicar o mesmo critério às restantes atividades, num total de 16 pontos. A pontuação orienta o estudo e não constitui classificação formal.